CONSTITUTION

INHERITED FROM Helix Constitution

This module is a submodule of a Helix-family project (e.g. HelixCode, HelixAgent, ATMOSphere) that includes the Helix Constitution submodule at the parent's `constitution/` path. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references (universal anti-bluff covenant §11.4, no-guessing mandate §11.4.6, credentials-handling mandate §11.4.10, host-session safety §12, data safety §9, mutation- paired gates §1.1) apply unconditionally to every change landed here. The module-specific rules below extend them — they never weaken any universal clause.

When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: https://github.com/HelixDevelopment/Helix-Constitution

---

Containers Module Constitution

INHERITED FROM constitution/Constitution.md

All rules in `constitution/Constitution.md` (and the `constitution/Constitution.md` it references) apply unconditionally. This file's rules below extend them — they MUST NOT weaken any inherited rule. See parent root `CLAUDE.md` §6.AD for the Lava-specific incorporation context (29th §6.L cycle, 2026-05-14) and §6.AD-debt for the implementation-gap inventory. Use `constitution/find_constitution.sh` from the parent project root to resolve the absolute path of the submodule from any nested location.

This Constitution governs the `digital.vasic.containers` module. It inherits the universal mandatory constraints cascaded from the HelixAgent root `CLAUDE.md` and applies them to this module's scope. Module-specific addenda are welcome but cannot weaken or override the universal rules.

Scope

`digital.vasic.containers` is a generic, project-agnostic Go module for container orchestration, health checking, lifecycle management, remote distribution, and service discovery across Docker, Podman, and Kubernetes runtimes. It is the single integration point through which the HelixAgent binary (and any other consumer) brings up its full container topology — local and remote — driven entirely by the consumer's `.env` file (`Containers/.env` for HelixAgent).

This module is foundational: it has no upstream sibling modules and is consumed by `Challenges`, `HelixLLM`, `HelixQA`, and HelixAgent itself.

Module-Specific Invariants

1. **Project-agnostic.** No hardcoded project-specific package names, endpoints, device serials, or fixtures. All consumer-specific data is registered via the public API. Default values are empty or generic.
2. **Sole orchestrator role.** The module's runtime is the only sanctioned path for container lifecycle operations. Direct `docker`/`podman` commands and `docker-compose up|down` are prohibited as workflows in any consumer.
3. **Dynamic remote-host enrolment (CONST-031).** Remote hosts are loaded from `CONTAINERS_REMOTE_HOST_N_*` env vars (N=1..100). The loader (`pkg/envconfig/parser.go`) stops at the first absent `_NAME`. No hostname is hardcoded anywhere else in the repo.
4. **Rootless / no sudo.** No `sudo` or `su` in source, scripts, tests, or docs. Use rootless container runtimes only.
5. **Health-check parity.** Every service started by this module must expose a TCP or HTTP health endpoint and pass `HealthChecker` checks with retries before being considered up.
6. **Rebuild-on-change.** Any code change affecting a containerised component requires rebuilding and redeploying the container locally, and re-running with `CONTAINERS_REMOTE_ENABLED=true` for remote distribution.

Universal Mandatory Constraints

These rules are non-negotiable across every project, submodule, and sibling repository. They are derived from the HelixAgent root `CLAUDE.md`. Each project MUST surface them in its own `CLAUDE.md`, `AGENTS.md`, and `CONSTITUTION.md`. Project-specific addenda are welcome but cannot weaken or override these.

Hard Stops (permanent, non-negotiable)

1. **NO CI/CD pipelines.** No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks either. All builds and tests run manually or via Makefile/ script targets.
2. **NO HTTPS for Git.** SSH URLs only (`git@github.com:…`, `git@gitlab.com:…`, etc.) for clones, fetches, pushes, and submodule updates. Including for public repos. SSH keys are configured on every service.
3. **NO manual container commands.** Container orchestration is owned by the project's binary/orchestrator (e.g. `make build` → `./bin/<app>`). Direct `docker/podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows. The orchestrator reads its configured `.env` and brings up everything.

Mandatory Development Standards

1. **100% Test Coverage.** Every component MUST have unit, integration, E2E, automation, security/penetration, and benchmark tests. No false positives. Mocks/stubs ONLY in unit tests; all other test types use real data and live services.
2. **Challenge Coverage.** Every component MUST have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success — validate actual behavior, not return codes.
3. **Real Data.** Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.
4. **Health & Observability.** Every service MUST expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.
5. **Documentation & Quality.** Update `CLAUDE.md`, `AGENTS.md`, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: `<type>(<scope>): <description>`.
6. **Validation Before Release.** Pass the project's full validation suite (`make ci-validate-all`-equivalent) plus all challenges (`./challenges/scripts/run_all_challenges.sh`).
7. **No Mocks or Stubs in Production.** Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully

functional with real integrations. Only unit tests may use mocks/stubs.

8. **Comprehensive Verification.** Every fix MUST be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives in tests or challenges. Grep-only validation is NEVER sufficient.
9. **Resource Limits for Tests & Challenges (CRITICAL).** ALL test and challenge execution MUST be strictly limited to 30-40% of host system resources. Use `GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3`, `-p 1` for `go test`. Container limits required. The host runs mission-critical processes — exceeding limits causes system crashes.
10. **Bugfix Documentation.** All bug fixes MUST be documented in `docs/issues/fixed/BUGFIXES.md` (or the project's equivalent) with root cause analysis, affected files, fix description, and a link to the verification test/challenge.
11. **Real Infrastructure for All Non-Unit Tests.** Mocks/fakes/ stubs/ placeholders MAY be used ONLY in unit tests (files ending `_test.go` run under `go test -short`, equivalent for other languages). ALL other test types — integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification — MUST execute against the REAL running system with REAL containers, REAL databases, REAL services, and REAL HTTP calls. Non-unit tests that cannot connect to real services MUST skip (not fail).
12. **Reproduction-Before-Fix (CONST-032 — MANDATORY).** Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence: (1) Write the Challenge first. (2) Run it; confirm fail (it reproduces the bug). (3) Then write the fix. (4) Re-run; confirm pass. (5) Commit Challenge + fix together. The Challenge becomes the regression guard for that bug forever.
13. **Concurrent-Safe Containers (Go-specific, where applicable).** Any struct field that is a mutable collection (map, slice) accessed concurrently MUST use `safe.Store[K,V]` / `safe.Slice[T]` from `digital.vasic.concurrency/pkg/safe` (or the project's equivalent primitives). Bare `sync.Mutex + map/slice` combinations are prohibited for new code.

Definition of Done (universal)

A change is NOT done because code compiles and tests pass. "Done" requires pasted terminal output from a real run, produced in the same session as the change.

- **No self-certification.** Words like *verified, tested, working, complete, fixed, passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.

- **Demo before code.** Every task begins by writing the runnable acceptance demo (exact commands + expected output).
- **Real system, every time.** Demos run against real artifacts.
- **Skips are loud.** `t.Skip` / `@Ignore` / `xit` / `describe.skip` without a trailing `SKIP-OK: #<ticket>` comment break validation.
- **Evidence in the PR.** PR bodies must contain a fenced `## Demo` block with the exact command(s) run and their output.

CONST-033 — Host Power Management is Forbidden

Status: Mandatory. Non-negotiable. Applies to every project, submodule, container entry point, build script, test, challenge, and systemd unit shipped from this repository.

Rule: No code in this repository may invoke a host-level power-state transition (suspend, hibernate, hybrid-sleep, suspend-then-hibernate, poweroff, halt, reboot, kexec) on the host machine. This includes — but is not limited to:

- `systemctl {suspend,hibernate,hybrid-sleep,suspend-then-hibernate,poweroff,halt,reboot,kexec}`
- `loginctl {suspend,hibernate,hybrid-sleep,suspend-then-hibernate,poweroff,halt,reboot}`
- `pm-{suspend,hibernate,suspend-hybrid}`
- `shutdown {-h,-r,-P,-H,now,--halt,--poweroff,--reboot}`
- DBus calls to `org.freedesktop.login1.Manager.{Suspend,Hibernate,HybridSleep,SuspendThenHibernate,PowerOff,Reboot}`
- DBus calls to `org.freedesktop.UPower.{Suspend,Hibernate,HybridSleep}`
- `gsettings set ... sleep-inactive-{ac,battery}-type` to any value other than `'nothing'` or `'blank'`

Why: The host runs mission-critical parallel CLI-agent and container workloads. On 2026-04-26 18:23:43 the host was auto- suspended by the GDM greeter's idle policy mid-session, killing HelixAgent and 41 dependent services. Recurring memory-pressure SIGKILLs of `user@1000.service` (perceived as "logged out") have the same outcome. Auto-suspend, hibernate, and any power-state transition are unsafe for this host.

Defence in depth (mandatory artifacts in every project):

1. `scripts/host-power-management/install-host-suspend-guard.sh` — privileged installer, manual prereq, run once per host with sudo. Masks `sleep.target`, `suspend.target`, `hibernate.target`, `hybrid-sleep.target`; writes `AllowSuspend=no` drop-in; sets logind `IdleAction=ignore` and `HandleLidSwitch=ignore`.
2. `scripts/host-power-management/user_session_no_suspend_bootstrap.sh` — per-user, no-sudo de-

fensive layer. Idempotent. Safe to source from `start.sh` / `setup.sh` / `bootstrap.sh`.
3. `scripts/host-power-management/check-no-suspend-calls.sh` — static scanner. Exits non-zero on any forbidden invocation.
4. `challenges/scripts/host_no_auto_suspend_challenge.sh` — asserts the running host's state matches layer-1 masking.
5. `challenges/scripts/no_suspend_calls_challenge.sh` — wraps the scanner as a challenge that runs in CI / `run_all_challenges.sh`.

Enforcement: Every project's CI / `run_all_challenges.sh` equivalent MUST run both challenges (host state + source tree). A violation in either channel blocks merge. Adding files to the scanner's `EXCLUDE_PATHS` requires an explicit justification comment identifying the non-host context.

See also: `docs/HOST_POWER_MANAGEMENT.md` for full background and runbook.

MANDATORY HOST-SESSION SAFETY (Constitution §12)

Forensic incident, 2026-04-27 22:22:14 (MSK): the developer's `user@1000.service` was SIGKILLed under an OOM cascade triggered by `pip3 install --user openai-whisper` running on top of chronic podman-pod memory pressure. The cascade SIGKILLed gnome-shell, every ssh session, claude-code, tmux, btop, npm, node, java, pip3 — full session loss. Evidence: `journalctl --since "2026-04-27 22:00" --until "2026-04-27 22:23"`.

This invariant applies to **every script, test, helper, and AI agent** in this submodule. Non-compliance is a release blocker.

Forbidden — directly OR indirectly

1. **Suspending the host**: `systemctl suspend`, `pm-suspend`, `loginctl suspend`, DBus `org.freedesktop.login1.Suspend`, GNOME idle-suspend, lid-close handler.
2. **Hibernating / hybrid-sleeping**: any `Hibernate` / `HybridSleep` / `SuspendThenHibernate` method.
3. **Logging out the user**: `loginctl terminate-session`, `pkill -u <user>`, `systemctl --user --kill`, anything that signals `user@<uid>.service`.
4. **Unbounded-memory operations** inside `user@<uid>.service` cgroup. Any single command expected to exceed 4 GB RSS MUST be wrapped in `bounded_run` (defined in `scripts/lib/host_session_safety.sh`, parent repo).
5. **Programmatic rfkill toggles, lid-switch handlers, or power-button handlers** — these cascade into idle-actions.

6. **Disabling systemd-logind, GDM, or session managers** "to make things faster" — even temporary stops leave the system unable to recover the user session.

Required safeguards

Every script in this submodule that performs heavy work (build, transcription, model inference, large compression, multi-GB git op) MUST:

1. Source `scripts/lib/host_session_safety.sh` from the parent repo.
2. Call `host_check_safety` at the top and **abort if it fails**.
3. Wrap any subprocess expected to exceed ~4 GB RSS in `bounded_run "<name>" <max-mem> <max-time> -- <cmd...>` so the kernel OOM killer is contained to that scope and cannot escalate to user.slice.
4. Cap parallelism (`-j`) to fit available RAM (each AOSP job ≈ 5 GB peak RSS).

Container hygiene

Containers (Docker / Podman) we own or rely on MUST:

1. Declare an explicit memory limit (`mem_limit` / `--memory` / `MemoryMax`).
2. Set `OOMPolicy=stop` in their systemd unit to avoid retry loops.
3. Use exponential-backoff restart policies, never immediate retry.
4. Be clean-slate destroyed (`podman pod stop && rm`, `podman volume prune`) and rebuilt after any host crash or session loss so stale lock files don't keep producing failures.

When in doubt

Don't run heavy work blind. Check `journalctl -k --since "1 hour ago" | grep -c oom-kill`. If it's non-zero, **fix the offending workload first**. Do not stack new work on a host already in distress.

Cross-reference: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §12 (full forensic, library API, operator directives) + parent `scripts/lib/host_session_safety.sh`.

MANDATORY ANTI-BLUFF VALIDATION (Constitution §8.1 + §11)

This submodule inherits the parent ATMOSphere project's anti-bluff covenant. A test that PASSes while the feature it claims to validate is unusable to an end user is the single most damaging failure mode in this codebase. It has shipped working-on-paper / broken-on-device builds before, and that MUST NOT happen again.

The canonical authority is `docs/guides/ATMOSPHERE_CONSTITUTION.md` §8.1 ("NO BLUFF — positive-evidence-only validation") and §11 ("Bleeding-edge ultra-perfection") in the parent repo. Every contribution to THIS submodule is bound by it. Summarised non-negotiables:

1. **Tests MUST validate user-visible behaviour, not just metadata.** A gate that greps for a string in a config XML, an XML attribute, a manifest entry, or a build-time symbol is METADATA — not evidence the feature works for the end user. Such a gate is allowed ONLY when paired with a runtime / on-device test that exercises the user-visible path and reads POSITIVE EVIDENCE that the behaviour actually occurred (kernel `/proc/*` runtime state, captured audio/video, dumpsys output produced *during* playback, real input-event delivery, real surface composition, etc).
2. **PASS / FAIL / SKIP must be mechanically distinguishable.** SKIP is for environment limitations (no HDMI sink, no USB mic, geo-restricted endpoint unreachable) and MUST always carry an explicit reason. PASS is reserved for cases where positive evidence was observed. A test that completes without observing evidence MUST NOT report PASS.
3. **Every gate MUST have a paired mutation test in `scripts/testing/meta_test_false_positive_proof.sh` (parent repo).** The mutation deliberately breaks the feature and the gate MUST then FAIL. A gate without a paired mutation is a BLUFF gate and is a Constitution violation regardless of how many checks it appears to make.
4. **Challenges (HelixQA) and tests are in the same boat.** A Challenge that reports "completed" by checking the test runner exited 0, without observing the system behaviour the Challenge is supposed to verify, is a bluff. Challenge runners MUST cross-reference real device telemetry (logcat, captured frames, network probes, kernel state) to confirm the user-visible promise was kept.
5. **The bar for shipping is not "tests pass" but "users can use the feature."** If the on-device experience does not match what the test claims, the test is the bug. Fix the test (positive-evidence harder), do not silence it.
6. **No false-success results are tolerable.** A green test suite combined with a broken feature is a worse outcome than an

honest red one — it silently destroys trust in the entire suite. Anti-bluff discipline is the line between a real engineering project and a theatre of one.

When in doubt: capture runtime evidence, attach it to the test result, and let a hostile reviewer (i.e. yourself, in six months) try to disprove that the feature really worked. If they can, the test is bluff and must be hardened.

Cross-references: parent CLAUDE.md "MANDATORY DEVELOPMENT PRINCIPLES", parent AGENTS.md "NO BLUFF" section, parent `scripts/testing/meta_test_false_positive_proof.sh`.

Seventh Law inheritance (Anti-Bluff Enforcement, 2026-04-30)

In addition to the Sixth Law above, this submodule inherits Lava's **Seventh Law — Tests MUST Confirm User-Reachable Functionality (Anti-Bluff Enforcement)** when consumed by the Lava project (`vasic-digital/Lava`). The Seventh Law was added to Lava's `CLAUDE.md` on 2026-04-30 to mechanically enforce the Sixth Law: every test commit MUST carry a Bluff-Audit stamp (mutation/observed-failure/reverted protocol); every feature MUST pass a real-stack verification gate; release tags MUST be preceded by a real-device attestation; forbidden test patterns (mocking the SUT, verification-only assertions, ignored tests without follow-up, build-success-as-only-assertion) are pre-push-rejected; a recurring bluff hunt and a bluff discovery protocol apply.

The authoritative verbatim text lives in the parent Lava `CLAUDE.md` under "Seventh Law — Tests MUST Confirm User-Reachable Functionality (Anti-Bluff Enforcement)". This submodule MAY add stricter clauses but MUST NOT relax any of the seven Seventh-Law clauses. Both the submodule's own anti-bluff rules and Lava's Sixth + Seventh Laws are binding when consumed by Lava; the stricter of the two applies.

Clause 6.L — Anti-Bluff Functional Reality Mandate (Operator's Standing Order)

Inherited verbatim from parent Lava `/CLAUDE.md` §6.L. The operator has invoked this mandate **TWENTY-THREE TIMES** across two working days. The 10th invocation (2026-05-05, after Phase 7 readiness was reported, when the operator commissioned the full rebuild-and-test-everything cycle for tag Lava-Android-1.2.3): "Rebuild Go API and client app(s), put new builds into releases dir (with properly updated version codes) and execute all existing

tests and Challenges! Any issue that pops up MUST BE properly addressed by addressing the root causes (fixing them) and covering everything with validation and verification tests and Challenges!"

Every test, every Challenge Test, every CI gate added to or maintained in this submodule MUST do exactly one job: confirm the feature it claims to cover actually works for an end user, end-to-end, on the gating matrix. CI green is necessary, NEVER sufficient. Tests must guarantee the product works — anything else is theatre.

Inheritance is recursive. Sub-submodules MAY paste this clause verbatim; they MUST NOT abbreviate or relax it.

Clause 6.O (added 2026-05-05, inherited per 6.F)

- **Clause 6.O — Crashlytics-Resolved Issue Coverage Mandate** — see root `/CLAUDE.md` §6.O. Every Crashlytics-recorded issue (fatal OR non-fatal) closed/resolved by any commit MUST gain (a) a validation test in the language of the crashing surface that reproduces the conditions, (b) a Challenge Test under `app/src/androidTest/kotlin/lava/app/challenges/` (client) or `tests/e2e/` (server) that drives the same user-facing path, and (c) a closure log at `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` recording the issue ID, root-cause analysis, fix commit SHA, and links to the tests. `scripts/tag.sh` MUST refuse release tags whose CHANGELOG mentions Crashlytics fixes without matching closure logs. Marking a Crashlytics issue "closed" in the Console requires the test coverage to land first — never close-mark before the regression-immunity tests exist. Forensic anchor: 2026-05-05, 2 Crashlytics-recorded crashes within minutes of the first Firebase-instrumented APK distribution (Lava-Android-1.2.3-1023, commit `e9de508`); postmortem at `.lava-ci-evidence/crashlytics-resolved/2026-05-05-firebase-init-hardening.md`. The operator's ELEVENTH §6.L invocation made this clause load-bearing.

Clause 6.P (added 2026-05-05, inherited per 6.F)

- **Clause 6.P — Distribution Versioning + Changelog Mandate** — see root `/CLAUDE.md` §6.P. Every distribute action (Firebase App Distribution, container registry pushes, releases/snapshots, scripts/tag.sh) MUST: (1) carry a strictly increasing versionCode (no re-distribution of already-published codes); (2) include a CHANGELOG entry — canonical file `CHANGELOG.md`

at repo root + per-version snapshot at `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>.md`; (3) inject the changelog into the App Distribution release-notes via `--release-notes`. `scripts/firebase-distribute.sh` REFUSES to operate when current versionCode ≤ last-distributed versionCode for the channel, OR when CHANGELOG.md lacks an entry for the current version, OR when the per-version snapshot file is missing. `scripts/tag.sh` enforces the same gates pre-tag. Re-distributing the same versionCode is forbidden across distribute sessions; idempotent retry within a single session is permitted. Forensic anchor: 2026-05-05 23:11 operator's TWELFTH §6.L invocation: "when distributing new build it must have version code bigger by at least one then the last version code available for download (already distribited). Every distributed build MUST CONTAIN changelog with the details what it includes compared to previous one we have published!"

Clause 6.Q (added 2026-05-05, inherited per 6.F)

- **Clause 6.Q — Compose Layout Antipattern Guard** — see root `/CLAUDE.md` §6.Q. Forbids nesting vertically-scrolling lazy layouts (LazyColumn, LazyVerticalGrid, LazyVerticalStaggeredGrid) inside parents giving unbounded vertical space (verticalScroll, unbounded wrapContentHeight, LinearLayout-with-weight wrapper). Equivalent rule horizontally for LazyRow / LazyHorizontalGrid / LazyHorizontalStaggeredGrid. Per-feature structural tests + Compose UI Challenge Tests on the §6.I matrix are the load-bearing acceptance gates. Forensic anchor: 2026-05-05 23:51 operator-reported "Opening Trackers from Settings crashes the app" — TrackerSelectorList used LazyColumn nested in TrackerSettingsScreen's Column(verticalScroll). Closure log: `.lava-ci-evidence/crashlytics-resolved/2026-05-05-tracker-settings-nested-scroll.md`. Pattern guard: `feature/tracker_settings/src/test/.../TrackerSelectorListLazyColumnRegressionTest.kt`. The operator THIRTEENTH §6.L invocation triggered this clause.

Article XI §11.9 — Anti-Bluff Forensic Anchor (CONST-035) — cascaded from HelixCode root

> Verbatim user mandate: "*We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be*

> *used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"*

Operative rule: The bar for shipping is not "tests pass" but "users can use the feature." Every PASS in this codebase MUST carry positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects regardless of how green the summary line looks. No false-success results are tolerable.

Repository scope: This anchor applies to all tests, all Challenges, and all CI/CD validation in this repository. It is cascaded from HelixCode root CONSTITUTION.md / CLAUDE.md / AGENTS.md and is identical across the HelixDevelopment + vasic-digital organizations.

CONST-036 — Continuation Document Maintenance Mandate

`docs/CONTINUATION.md` MUST be the single-file source-of-truth handoff document for resuming work across any CLI session. Every commit that changes phase status, lands a new spec/plan, bumps a submodule pin, ships a release artifact, discovers/resolves a known issue, or implements an operator scope directive MUST update `docs/CONTINUATION.md` in the SAME COMMIT. The "Last updated" line MUST track HEAD. See root `CLAUDE.md` §6.S for the inherited clause text.

CONST-042 — No-Secret-Leak (cascaded)

No credential may be committed. All secrets in .env (gitignored). Any leak is a release blocker.

CONST-043 — No-Force-Push (cascaded)

No force push, history rewrite, branch deletion of main without explicit per-operation user approval.

§6.R — No-Hardcoding Mandate (inherited 2026-05-06, per §6.F)

See root `/CLAUDE.md` §6.R. No connection address, port, header field name, credential, key, salt, secret, schedule, algorithm parameter, or domain literal shall appear as a string/int constant in tracked source code. Every such value MUST come from `.env` (gitignored), generated config class, runtime env var, or mounted file. This submodule MAY add stricter rules but MUST NOT relax.

§6.T — Universal Quality Constraints (inherited 2026-05-06, per §6.F)

See root `/CLAUDE.md` §6.T. All four sub-points (Reproduction-Before-Fix, Resource Limits for Tests & Challenges, No-Force-Push, Bugfix Documentation) apply verbatim. This submodule MAY add stricter rules but MUST NOT relax any of §6.T.1–§6.T.4.

§6.U — No sudo/su Mandate (inherited 2026-05-08, per §6.F)

See root `/CLAUDE.md` §6.U. Every use of `sudo` or `su` is strictly forbidden. Operations requiring elevated privileges MUST use container-based solutions from the `vasic-digital/Containers` submodule or be provided by local project/Submodule dependencies that build automatically. The pre-push hook rejects files containing `sudo` or `su` patterns. This submodule MAY add stricter rules but MUST NOT relax.

§6.V — Container Emulators Mandate (inherited 2026-05-08, per §6.F)

See root `/CLAUDE.md` §6.V. Every Android emulator instance for Challenge Tests / UI verification MUST run inside a container managed by the `vasic-digital/Containers` submodule. Rootless Podman/Docker only. All tests execute inside containers. The §6.I matrix (API 28/30/34/latest, phone/tablet/TV) runs inside container-bound emulators. This submodule MAY add stricter rules but MUST NOT relax.

§6.W — GitHub + GitLab Only Remotes (inherited 2026-05-08, per §6.F)

See root `/CLAUDE.md` §6.W. Only GitHub (`vasic-digital/*`, `HelixDevelopment/*`) and GitLab (`vasic-digital/*`, `HelixDevelopment/*`) are permitted as Git remotes. GitFlic, GitVerse, and all other providers are forbidden. The 4-mirror model is replaced by 2-mirror (GitHub + GitLab). This submodule MAY add stricter rules but MUST NOT relax.

§6.X — Container-Submodule Emulator Wiring Mandate (inherited 2026-05-13, per §6.F)

See root `/CLAUDE.md` §6.X. Every Android emulator instance the project depends on for testing MUST execute its emulator process INSIDE a podman/docker container managed by `Submodules/Containers/`, NOT be host-direct-launched by Containers-submodule code that runs on the host. The Containers submodule's `pkg/runtime/` (rootless podman/docker auto-detection) brings the container up; `pkg/emulator/` orchestrates the AVD lifecycle inside it. Lava-side `scripts/run-emulator-tests.sh` is thin glue forwarding to the Containers CLI. The container-bound path is the gate — host-direct emulators are permitted for workstation iteration only. §6.X-debt tracks the wiring implementation owed to `Submodules/Containers/`. This submodule MAY add stricter rules but MUST NOT relax.

§11.4.7 — Operator-Path Test Coverage Rule (inherited from vasic-digital/tmux, 2026-05-13)

Forensic anchor. Caught in `vasic-digital/tmux` 2026-05-13: tests that reported GREEN while the operator-facing feature was broken, because the tests bypassed the operator's wrapper and hand-crafted the underlying primitives (systemd-run scopes) directly. Two stacked failure modes:

- **Test 11** always passed `-S "$SOCKET"` → exercised the explicit-socket path only; the operator's `tmx new -s X` (default socket) was uncovered. The captured-evidence claim was partial. Operator reported the bug visually: status bar showed default green instead of the hostname-derived colour. README marketing was a §1 bluff.

- **Test 14** hand-spawned three `systemd-run --user --scope` units with explicit `--unit` names to simulate isolation. The actual `tmx new -s A -d; tmx new -s B -d` placed every session in ONE shared cgroup scope. Test 14 PASSed; operator-facing isolation did not exist. README's "if one session OOMs, others survive" was a §1 bluff.

The mandate. Every gate test for a feature MUST exercise the SAME entry point an end-user would invoke in production. Tests that bypass the operator's wrapper, helper, or install path — and instead reproduce its effects with hand-crafted equivalents — DO NOT satisfy captured-evidence requirements. When the operator's path and the test's path diverge:

1. The test header MUST EXPLICITLY name what divergence exists.
2. A SEPARATE end-to-end test MUST close that divergence with captured evidence on the operator-facing entry point.

Operative test. For every test under this submodule's `scripts/` `anti-bluff/` / `tests/` / `cmd/distributed-test/` paths, ask: "would a consumer of `digital.vasic.containers` hit this code path in their normal workflow?" If no, that test is supplementary; the operator-path test must exist alongside it.

No grep-on-script-content alone. A `grep` for a literal flag or property name inside the wrapper/orchestrator/script is allowed AS A STATIC CHECK in addition to a runtime readback — never as the only assertion.

Layer-4 mutations MUST target operator-path code, not synthetic- test code. When the submodule provides BOTH a thin host-side bridge (e.g. `scripts/tmx` on Darwin) AND a thick body of behaviour (e.g. `scripts/tmx-vm` inside the VM), paired mutations MUST target the body — that's the file consumers actually exercise. Mutating the bridge while the consumer-path lives in the body is a §1 bluff inside the gate itself.

Inheritance and propagation. This submodule's tests inherit §11.4.7 from the parent `vasic-digital/tmux` Constitution. Submodule rules MAY add stricter clauses but MUST NOT relax. Both the parent project's rule set and this submodule's own apply; the stricter applies.

Non-compliance is a release blocker.

Submodule Decoupling & Reusability — Mandatory

Status: Mandatory. Non-negotiable.

Rule: This repository is a **shared submodule** consumed by multiple independent consumer projects. Its value depends on staying **fully decoupled and reusable**. No change in this repository may introduce coupling that breaks its standalone reusability for any consumer.

Prohibited inside this repository:

1. Hardcoding any specific consumer project's name, paths, platform list, version strings, release-naming conventions, branding, or feature names.
2. `import` / dependency on any consumer-project namespace, package, or build coordinate.
3. Embedding consumer-project-specific governance, rule numbering, or release cadence into this repository's `CONSTITUTION.md` / `CLAUDE.md` / `AGENTS.md`.
4. Assuming this repository is consumed by a particular CLI, build system, language toolchain version, or target architecture beyond what its public interface documents.

Required inside this repository:

1. All public surfaces (APIs, CLIs, configuration files, environment variables, scripts) MUST be expressed in terms of THIS repository's own domain — not any consumer's.
2. Governance MUST describe responsibilities and contract from THIS repository's perspective. Consumer projects appear as illustrative examples at most, never as load-bearing requirements.
3. Cross-project rules adopted from a consumer (such as a cross-platform impact mandate) MUST be phrased generically — "every consuming project's full platform matrix" — and never hardcode any single consumer's matrix.

Why: Repositories like this one have shipped changes in the past where one consumer's platform list, feature names, or rule numbering leaked into shared-repo governance — and then collided at merge time with another consumer's parallel work, leaving the repository unmergeable until manual conflict resolution stripped the consumer-specific text back out. Decoupling is the only mechanism that preserves this repository's value as shared infrastructure.

Recursive scope: any submodule this repository consumes inherits the same decoupling+reusability rule. Third-party upstream submodules that this repository merely vendors (e.g. open-source tools under a `tools/opensource/` tree, if present) are explicitly out of scope — we are not their owners.

CONST-047 — Recursive Submodule Application Mandate (cascaded from root CONSTITUTION.md)

> Verbatim user mandate (2026-05-14): "*Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!*"

Every engineering deliverable produced for the main project MUST be applied — fully and recursively — to every owned submodule under the `vasic-digital` and `HelixDevelopment` GitHub organizations. Each owned submodule (including this one) MUST receive in lockstep: (1) anti-bluff posture (CONST-035 / Article XI §11.9), (2) comprehensive documentation matching actual capabilities, (3) full tests + Challenges coverage with captured runtime evidence, (4) recursive propagation through nested submodules under the same orgs, (5) synchronized commits when meta-repo state advances this surface.

See the root `CONSTITUTION.md` §CONST-047 for the full mandate. This anchor MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

Cross-Platform Impact — Mandatory Consideration (mirrors Yole CONST-037)

Status: Mandatory. Non-negotiable. Mirrors CONST-037 in the parent Yole project's `CONSTITUTION.md`.

Rule: This submodule is consumed by the Yole multi-platform project (Android / Desktop / iOS / Web). Every change MUST be reasoned about across all four target platforms BEFORE coding. A fix that works on one target but silently breaks another is a regression.

Pre-edit checklist: Before any code change, answer:

1. Does this compile on every Yole target (Android, Desktop, iOS, Web)?
2. Does it behave identically — or by-design differently — on each?
3. Is the change covered by a test on every affected target?
4. Are platform manifests (AndroidManifest.xml, Info.plist, web manifest, container packaging) updated coherently?

Commit body requirement: every change MUST include a "Cross-platform impact" block listing each Yole platform's disposition (changed / unchanged / N/A with reason).

```
Cross-platform impact:
- Android: <disposition>
- Desktop: <disposition>
- iOS:     <disposition>
- Web:     <disposition>
```

Why: End users experience the integrated Yole product, not this submodule in isolation. Cross-platform regressions caused by submodule-local changes have shipped to users in the past; mandatory up-front consideration is the only mitigation.

Enforcement: the parent Yole repo runs `yole-challenges/scripts/cross_platform_parity_challenge.sh` in `make qa-all`. Submodule changes that cause that challenge to fail MUST be reverted or fixed.

See also: CONST-037 in the parent Yole repo's `CONSTITUTION.md` for the full rule and forensic anchor.

§6.Z — Anti-Bluff Distribute Guard (inherited 2026-05-14, per §6.F)

See root `/CLAUDE.md` §6.Z. No artifact may be distributed (Firebase App Distribution, Google Play Store release, container image push, this submodule's binary release, any future channel) UNLESS the corresponding end-to-end tests have been **EXECUTED — not source-compiled, EXECUTED** — against the EXACT artifact about to be distributed, AND have **passed**. Pre-distribute test-evidence file required at `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>-test-evidence.{md,json}` with matching commit SHA, timestamp within 24h, `BUILD SUCCESSFUL` (or per-language pass marker) verbatim in captured output. Cold-start verification is the load-bearing canary. Distributing a faulty version is a constitutional violation by construction. §6.Z-debt is open: mechanical enforcement via `scripts/firebase-distribute.sh` Phase 1 Gate 6 + pre-push hook check is documented but not yet enforced. Forensic anchor: 2026-05-14 Galaxy S23 Ultra cold-launch crash on Lava-Android-1.2.19-1039 (Crashlytics `40a62f97a5c65abb56142b4ca2c37eeb` — `painterResource()` rejection of `<layer-list>` drawable); agent had skipped Compose UI test execution citing the wrong §6.X caveat. Operator's 26th §6.L invocation: "Anti-bluff policy MUST BE ENFORCED ALWAYS!!!" This submodule MAY add stricter rules but MUST NOT relax this clause.

§6.AA — Two-Stage Distribute Mandate (inherited 2026-05-14, per §6.F)

See root `/CLAUDE.md` §6.AA. When an artifact has both a debug and a release variant (or analogous dev-vs-prod build types — including this submodule's binary release if it ships separate dev / prod variants), distribute MUST happen in TWO STAGES with operator-confirmed verification between them. Stage 1 distributes the debug / dev variant only; the operator verifies the **distributed** debug variant on the failure-surface device class. Stage 2 distributes the release / prod variant only ONLY AFTER written stage-1 verification, with the §6.Z test-evidence file appended with a `release-stage` section. No combined distribute permitted by default; the combined path requires explicit per-cycle operator authorization recorded in the evidence file. The R8 / minification surprise class on Android (or analogous stripping / production-only optimization classes on other artifacts) is the load-bearing reason. §6.AA-debt is open: mechanical enforcement via `scripts/firebase-distribute.sh` default flip + refusal of out-of-order `--release-only` + paired `last-version-{debug,release}` per-channel pre-push check is documented but not yet enforced. Forensic anchor: 2026-05-14 operator directive immediately after the §6.Z forensic-anchor crash on Lava-Android-1.2.19-1039: "for purposes like this one we shall distribute via Firebase DEV / DEBUG version only. Once we try it, you continue and once all verified you distribute RELEASE too!" This submodule MAY add stricter rules but MUST NOT relax this clause.

§6.AB — Anti-Bluff Test-Suite Reinforcement (inherited 2026-05-14, per §6.F)

See root `/CLAUDE.md` §6.AB. Every existing test + Challenge in this submodule MUST be auditable for the anti-bluff property "would this test fail if the user-visible behavior broke in a way a real user would notice?" Per-feature completeness checklist: rendering correctness (assert dominant color matches expected hue, not just RGB-variance), state-machine completeness (negative tests for forbidden transitions), gating logic (gate fires only on actual completion criterion). Bluff-hunt cadence escalation: every defect not caught by an existing test triggers a 5-file defect-driven hunt of adjacent tests, recorded under `.lava-ci-evidence/bluff-hunt/<date>-defect-driven-<slug>.json`. Discrimination test mandatory per Challenge Test: deliberately-broken-but-non-crashing production code MUST cause the Challenge Test to fail. Forensic anchor: 2026-05-14 Lava-Android-1.2.20-1040 white-icon + onboarding-gate-bypass — both passed all existing tests but failed for the user. Operator's 27th §6.L invocation: "all existing tests and Chal-

lenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!" This submodule MAY add stricter rules but MUST NOT relax this clause.

§6.AC — Comprehensive Non-Fatal Telemetry Mandate (inherited 2026-05-14, per §6.F)

See root `/CLAUDE.md` §6.AC. Every catch / error / fallback / unexpected-state path on every distributable artifact in this submodule MUST record a non-fatal telemetry event with sufficient context to triage the failure remotely. The Android-side canonical entry is `analytics.recordNonFatal(throwable, ctx)` / `analytics.recordWarning(message, ctx)` (lava.common.analytics.AnalyticsTracker); the Go-side equivalent is `observability.RecordNonFatal(ctx, err, attrs)`. Cancellation throwables are filtered automatically. Mandatory context: feature/module + operation + error_class + error_message (truncated 1024, no credentials per §6.H) + per-platform extras. Forbidden: silent fallbacks without telemetry; credentials/tokens/cookies/PII unredacted in event attributes. §6.AC-debt is open: Detekt + Go-vet rules flagging `try/catch` blocks lacking the telemetry call, pre-push hook integration. Forensic anchor: 2026-05-14 operator: "Add comprehensive Crashlytics non-fatals recording all over the apps and API so we can track in the background all warnings, issues and unexpected situations!" This submodule MAY add stricter rules but MUST NOT relax this clause.

CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

> Verbatim user mandate (2026-05-15): "*Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!*"

No feature / functionality / flow / use case / edge case / service / application on any supported platform of this submodule is deliverable until covered by automation tests proving six invariants: (1) anti-bluff posture with captured runtime evidence (CONST-035); (2) proof of working capability end-to-end on target topology; (3) implementation matching documented promise; (4) no open issues/bugs surfaced; (5) full documentation in sync; (6) four-layer test floor (pre-build + post-build + runtime + paired mutation).

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-048 and constitution submodule `Constitution.md` §11.4.25 for the full mandate.

CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

> Verbatim user mandate (2026-05-15): "*Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!*"

Before ANY modification to `constitution/{Constitution,CLAUDE,AGENTS}.md` in the parent project, the agent or operator MUST execute the 7-step pipeline: (1) fetch + pull first

inside the constitution submodule worktree; (2) apply the change with §11.4.17 classification + verbatim mandate quote; (3) validate (meta-test + no merge-conflict markers + cross-file consistency); (4) commit + push to EVERY configured upstream of the constitution submodule (governance files only — never `git add -A`); (5) careful conflict resolution preserving union of governance content (force-push forbidden per CONST-043 / §9.2); (6) post-merge `git submodule update --remote --init` + re-run cascade verifier (CONST-047); (7) bump consuming project's `.gitmodules` pointer to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST-049 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-049 and constitution submodule `Constitution.md` §11.4.26 for the full mandate.

CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

> Verbatim user mandate (2026-05-15): "*Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — https://github.com/vasic-digital/Challenges). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — https://github.com/HelixDevelopment/HelixQA). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).*"

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources.

Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise this submodule's real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/Challenges submodule fully incorporated), HelixQA (HelixDevelopment/HelixQA submodule fully incorporated, with full autonomous QA sessions executing every registered test bank with captured wire evidence).

Required dependency submodules (recursive per CONST-047): Challenges + HelixQA + any other functionality submodules under vasic-digital/HelixDevelopment orgs this submodule depends on.

Cascade requirement: This anchor (verbatim or by CONST-050 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-050 and constitution submodule `Constitution.md` §11.4.27 for the full mandate.

CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

> Verbatim user mandate (2026-05-15): "*All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of*

> *the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!*"

Three cooperating invariants apply to every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, plus any subsequently authorised org — discoverable via `gh org list` / `glab`):

(A) Equal-codebase. This submodule is an EQUAL part of every consuming project's codebase. The consuming project's engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to this submodule on equal basis. Coverage ledgers (CONST-048) list this submodule as an in-scope target.

(B) Decoupling / reusability. This submodule MUST remain fully decoupled from any specific consuming project. NEVER inject project-specific context (hardcoded paths, hostnames, asset names, naming schemes). Stay project-not-aware, reusable, modular, completely testable as a standalone repository. When parent-project info is needed, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

(C) Dependency-layout. Any dependency this submodule consumes MUST be accessible from the consuming project's root at `<root>/<name>/` or `<root>/submodules/<name>/`. **Nested own-org submodule chains are FORBIDDEN** — this submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Third-party submodules are exempt.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-051 and constitution submodule `Constitution.md` §11.4.28 for the full mandate.

CONST-052: Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29)

> Verbatim user mandate (2026-05-15): "*naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of '_' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE re-*

> *named! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! ... There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. ... Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in out paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!*"

Every directory, submodule, and file in this submodule MUST use lowercase snake_case names. Existing non-compliant names MUST be renamed atomically with updates to every reference (configs, docs, source-code imports, governance files). Reference drift after rename = CONST-052 violation of equal severity to the rename itself.

Common-sense exceptions (technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE language-roots; vendor/upstream third-party submodules keep upstream names; build artefacts (`node_modules`, `__pycache__`, `.git`, `target`, `build`, `bin`) keep tool-mandated names. The test "does renaming break the technology?" trumps the rule.

`Upstreams/` → `upstreams/` transition: the constitution submodule's `install_upstreams.sh` (exported via `.bashrc`/`.zshrc`) supports BOTH directory layouts; lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): regression test for reference resolution + full test-type matrix run + anti-bluff wire-evidence captured.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-052 and constitution submodule `Constitution.md` §11.4.29 for the full mandate.

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

> Verbatim user mandate (2026-05-15): "*every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!*"

Every project module, owned-by-us submodule, service, and application MUST ship a proper `.gitignore`. Forbidden-from-version-control classes:

1. **Build artefacts**: `/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`, generator-produced files when the generator is committed.
2. **Cache files**: `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.cache/`, `.gradle/`, `.terraform/`, language-server caches.
3. **Temp files**: `*.tmp`, `*.swp`, `*~`, `.DS_Store`, `Thumbs.db`, `*.orig`, `*.rej`.
4. **Sensitive-data files**: `.env`, `.env.*` (allow `.env.example` placeholder only — no real secrets even as examples), `*.pem`, `*.key`, `*.crt`, `id_rsa*`, `id_ed25519*`, `.netrc`, `secrets/`, `api_keys.sh`.
5. **Generated reports/logs**: `*.log`, `coverage.out`, `htmlcov/`, runtime captures unless reference assets.
6. **OS/IDE personal state**: `.idea/`, `.history/`, `.vscode/` (except shared settings).

Anti-bluff invariant: `.gitignore` line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged` + `git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (unstage, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a `.env` leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by `CONST-053` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule `Constitution.md` §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

> Verbatim user mandate (2026-05-15): "*We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to veryone.*"

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}]`, `transitive_handling.{recursive,conflict_resolution}`, `language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/.helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by `CONST-054` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule `Constitution.md` §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

> Verbatim user mandate (2026-05-15): "*Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!*"

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke, etc.) against the post-pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: `Reopened`, Type: `Bug`); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by `CONST-055` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to

§11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule `Constitution.md` §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/add Mandate (cascaded from constitution submodule §11.4.36)

> Verbatim user mandate (2026-05-15): "*Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!*"

Every clone / add of a Git repository under HelixCode MUST be followed by `install_upstreams` invocation from the repository's root IF its tree contains `upstreams/` (or legacy `Upstreams/` per CONST-052 transition) populated with `*.sh` recipe files. The utility (installed on operator's `PATH` via `.bashrc`/`.zshrc`; implementation in the constitution submodule's `install_upstreams.sh` — already supports BOTH directory names since constitution commit `45d3678`) reads the recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs.

Skipping the invocation when `upstreams/` is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` + paired mutation. Automation: the future `incorporate-submodule` per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: `git remote -v | grep -c push` reports expected count.

Cascade requirement: This anchor (verbatim or by `CONST-056` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal `**Status:**` value, drawn from this 3-element closed map:

Item `**Type:**`	Closure `**Status:**` value
`Bug`	`Fixed (→ Fixed.md)`
`Feature`	`Implemented (→ Fixed.md)`
`Task`	`Completed (→ Fixed.md)`

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all "closed, positive evidence captured") while preserving the literal in the emitted document.

Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a CONST-057 violation. Gate `CM-CLOSURE-VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by `CONST-057` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** `AI` or `User` (source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).

- **Reason:** one-line cause classification — chosen from the closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }`. Other values permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The Issues_Summary.md Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`. Gate `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by `CONST-058` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule's** three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project's **repository-root files** (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md`) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md` import or the portable `## INHERITED FROM constitution/CLAUDE.md` heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: "the parent CLAUDE.md" / "the root Constitution" → constitution-submodule file at `constitution/<filename>`; "the project CLAUDE.md" / "this project's AGENTS.md" → consumer-side file at `<project-root>/<filename>`.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or explicit `Lifted from <project> to constitution per §11.4.35` / `Demoted from constitution to <project> per §11.4.35` commit-message annotation.

Gate `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by `CONST-059` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.35 for the full mandate.

CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

> Verbatim user mandate (2026-05-15): "*Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details.*"

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --
        quiet'
```

If `HEAD..@{u}` is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false**

confidence — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

Anti-bluff invariant: the fetch+log check MUST produce captured evidence — the actual `HEAD..@{u}` output, even if empty. Skipping the check on the basis of "I just fetched" or "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Cascade requirement: This anchor (verbatim or by `CONST-060` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule `Constitution.md` §11.4.37 for the full mandate.

CONST-062 — CONST-078: Round-191 cascade — anchors §11.4.42 through §11.4.58

The following anchors propagate from constitution submodule §11.4.42-58 (User mandates 2026-05-17 / 2026-05-18 / 2026-05-19) into this submodule's governance via short-form reference per CONST-049 step 6. Severity-equivalent to a §11.4 PASS-bluff at the respective enforcement layer. Cascade requirement: this block (verbatim or by ID reference) MUST appear in this submodule's CONSTITUTION.md, CLAUDE.md, AGENTS.md, and propagate to any nested owned-by-us submodule.

- **CONST-062 / §11.4.42 — Iteration-discipline mandate.** Each fix cycle MUST run pre-build gate + post-build/post-flash test + paired §1.1 mutation + post-validation §11.4.40 retest. Truncating cycle to ship faster = violation. Subagents default to this path.
- **CONST-063 / §11.4.43 — TDD-Fix-Discipline.** Every bug fix starts with a failing RED test reproducing the defect on the target topology. Fix lands only after RED → GREEN against real infrastructure. Skipping RED step = violation.
- **CONST-064 / §11.4.44 — Document Revision Header.** Every governance/status/plan doc MUST carry `**Revision:** N` + `**Last modified:** YYYY-MM-DD` + `**Maintainer:**` directly below H1; bumped per content edit. Missing/stale header = violation.
- **CONST-065 / §11.4.45 — Integration-Status-Doc Maintenance.** Every `Status.md` (integration, programme, sub-system) carries the §11.4.44 header and stays in sync with actual programme state at every commit advancing state. Out-of-sync = violation (CONST-044 severity).

- **CONST-066 / §11.4.46 — Validate-recent-work before post-flash sweep.** Each recent-work item since previous sweep MUST have its §11.4.43 RED test run against the live device and report GREEN before post-flash full-test sweeps. Skipping = violation.
- **CONST-067 / §11.4.47 — Firebase Data Review.** Every Firebase Crashlytics/Analytics finding triaged per §11.4.47 severity table, deduped against Issues.md; new stacktrace gets §11.4.43 RED test before fix. Untriaged past SLA = violation.
- **CONST-068 / §11.4.48 — UI-Driven Video Testing.** Every supported app/service ships UI-driven traversal tests covering surfaces A-E; each run captures screen-recording as anti-bluff evidence. Missing driver or missing video = violation.
- **CONST-069 / §11.4.49 — Dual-Approach Testing.** Every feature test ships in BOTH UI-driven (uiautomator) AND Intent-driven (programmatic API) variants. Both RED until fix lands, then both GREEN. Single-variant = violation.
- **CONST-070 / §11.4.50 — Deterministic Consistency.** Every test runs N iterations with consistent results; flaky tests forbidden; intermittent FAIL must be diagnosed not retried. First-PASS-only reporting = violation.
- **CONST-071 / §11.4.51 — Live-ADB-First Maximization.** Prefer live-ADB probes over post-flash sweeps wherever the fix surface allows (§11.4.43 step 2 maximised). Skipping live-probe when feasible = violation.
- **CONST-072 / §11.4.52 — Autonomous-Validation.** Every test/feature/flow MUST have an autonomous execution path (instrumentation APK / headless driver) so subagent flows run unattended. Operator-attended-only = violation.
- **CONST-073 / §11.4.53 — Fixed_Summary parity.** `Fixed_Summary.md` (+ HTML/PDF exports per §11.4.12/§11.4.44) regenerated on every Issues.md → Fixed.md migration. Stale exports = violation. HTML+PDF travel together with the `.md`.
- **CONST-074 / §11.4.54 — ATM-NNN ticket identifier.** Every Issue/Feature/Task entry carries an `ATM-NNN` id (zero-padded, monotonically increasing per project); cross-references in governance/plans/changelogs use the ATM-NNN id. Missing/duplicate = violation.
- **CONST-075 / §11.4.55 — Reopens-history + per-item Reopens.md.** Every Reopened item gets `docs/reopens/ATM-NNN.md` tracking each cycle (date, source AI/User, reason from CONST-058 vocabulary, evidence path). `Reopens_Summary.md` regenerated on every reopen. Missing per-item doc = violation.
- **CONST-076 / §11.4.56 — Status_Summary parity + two-audience format.** `Status_Summary.md` (+ HTML/PDF exports) ships in operator-side + AI-side sections and stays in parity with underlying status docs at every commit advancing state. Drift/missing audience section = violation.
- **CONST-077 / §11.4.57 — README.md doc-link section + revision metadata.** Every `README.md` carries (a) §11.4.44 revision header below H1, (b) Documentation link section listing

canonical governance + status + plan docs. Missing section or stale links = violation.
- **CONST-078 / §11.4.58 — Parallel-development methodology (PWU).** Project work proceeds through the Parallel Work Unit pipeline: Stage 1 DEVELOP (parallel, isolated worktrees) → Stage 2 MERGE (serial flock + §11.4.41 4-step) → Stage 3 REBUILD+FLASH → Stage 4 VALIDATE (parallel) → Stage 5 SWEEP. Four-layer lock hierarchy: L1 flock / L2 git / L3 contention-path advisory / L4 per-PWU worktree. Disjoint-scope PWUs run fully parallel; cross-scope overlap rejected to rebase. Conductor REFUSES merge of any PWU lacking captured evidence per §11.4.5/§11.4.52.

For full mandate text + verbatim user quotes + gates + paired mutations + composition tables, see constitution submodule `Constitution.md` §11.4.42-58 (canonical-root inheritance per CONST-059).

Round-207 cascade — §11.4.59 + §11.4.60 (README always-sync + Documentation always-sync composite covenant)

> Verbatim user mandate (2026-05-19): "*fully review and update our main README document. ... Make sure main README is among documents we MUST ALWAYS keep updated and in Sync with the projects and other documentation! Make sure we always export it (on every update) into PDF and HTML.*" AND (2026-05-19 ~09:00Z): "*Double check if all documents are properly tied with our root Constitution, CLAUDE.MD and AGENTS.MD so they are always up to date, always in sync and exported into PDF and HTML! ... Issues, Issues_Summary, Fixed, Fixed_Summary, Continuation, Status and Status_Summary for all contexts (areas) — THEY ALL MUST BE REGULARLY UPDATED, IN SYNC AND CONSISTENT without giving at any moment false picture about the state of the project or particular area(s) of it!*"

- **§11.4.59 — README always-sync mandate.** `README.md` at the project root is a §11.4.12-class always-sync document: kept current with every doc/integration/Status.md change, lockstep with `docs/CONTINUATION.md`, exported to `.html` + `.pdf` on every update via `scripts/testing/sync_readme_export.sh` (auto-invoked by `sync_issues_docs.sh`), carrying §11.4.44 revision header + Documentation Map section linking every Status / Status_Summary / spec / plan / guide / script-companion / changelog + the constitution submodule + per-audience navigation. Pre-build gate `CM-README-EXPORT-SYNC` enforces mtime parity (README.html + README.pdf ≥ README.md).

Paired mutation backdates HTML+PDF → gate FAILs. No escape hatch — no `--skip-readme-sync`, `--no-readme-export`, `--readme-stale-OK` flag.

- **§11.4.60 — Documentation always-sync composite covenant.** Eight doc classes (Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION, README, every Status.md, every Status_Summary.md) MUST be in sync at all times across `.md` + `.html` + `.pdf` artefacts. Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 govern individually; §11.4.60 binds them via single composite gate `CM-DOCS-COMPOSITE-SYNC` that FAILs the build if ANY instance's `.html` or `.pdf` mtime is older than `.md` mtime. Walks `docs/` recursively for Status fleet. Paired mutation backdates `docs/Issues.html` → gate FAILs. No escape hatch — no `--skip-composite-doc-sync`, `--allow-stale-html`, `--summary-not-applicable` flag exists.

Cascade requirement: These anchors (verbatim or by `§11.4.59` / `§11.4.60` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.59 + §11.4.60 for the full mandates.

CONST-068: Shell-script target-shell-parseability mandate (cascaded from constitution submodule §11.4.67)

> Verbatim user mandate (2026-05-19): "*any issue we spot must be fixed, bash scripts as well if they are broken!*" + "*Make sure that this is mandatory rule!*"

> Verbatim 2026-05-19 operator mandate: "*all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!*"

Every committed shell script MUST be parseable by its target interpreter (`sh -n` for `/bin/sh`, `bash -n` for `/bin/bash`, etc.) AND MUST declare a shebang matching its actual syntax usage. Bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, `<<<`, arrays, `${var^^}`, etc.) used in scripts that may be invoked via `sh script.sh` MUST be wrapped in `eval` so the parser sees only a string (target shells like mksh parse the entire script before executing — runtime guards cannot save a parse-time rejection). Honest shebangs only: `#!/bin/bash` only if bash actually expected; `#!/bin/sh` requires POSIX-clean body. Fix at source per §11.4.1, never at callsites. Composes with

§11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51. Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Cascade requirement: This anchor (verbatim or by `CONST-068` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.67 for the full mandate.

§11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

> Verbatim user mandate (2026-05-20): "*We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!*"

A test that asserts audio or video routing PASS MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing PASS: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from `/proc/asound/.../status hw_ptr`; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) tinycap RMS amplitude above the line-level floor. Empty / `<unreachable>` / `<N.E.>` / `<None>` placeholders are NOT positive evidence; a missing-but-required sink is `OPERATOR-BLOCKED` (release-blocker), never SKIP, never PASS. No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag exists.

Cascade requirement: This anchor (verbatim or by `§11.4.68` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.68 for the full mandate.

§11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

> Verbatim user mandate (2026-05-20): "*Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!*"

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

Cascade requirement: This anchor (verbatim or by `§11.4.70` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.70 for the full mandate.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

> Verbatim user mandate (2026-05-20): "*before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brough in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its*

> *success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!*"

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (`git fetch --all --prune --tags`, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto-`--ours`/`--theirs`); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit's body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with §11.4.4(b) four-layer coverage + §11.4.43 TDD-fix discipline, every PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with `git ls-remote` post-push. No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.71` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the push-discipline layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.71 for the full mandate.

§11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

> Verbatim user mandate (2026-05-20): "*Make sure all fixes for audio are always top priority in main working stream!*"

The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no "but this non-audio task is faster" or "but this research is more interesting" override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

Cascade requirement: This anchor (verbatim or by `§11.4.72` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a

process violation at the dispatch-priority layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.72 for the full mandate.

§11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

> Verbatim user mandate (2026-05-20): "*Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are appling here + revision and other properties we have!*"

Applies **only when a project recognises a main specification document**. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table `Revision` integer, bumped for every other change); (3) the metadata table MUST stay current (`Revision`, `Last modified`, `Status summary`, `Fixed`); (4) propagated copies of the rule MUST reference the active `specification.V<primary>.md`, not a stale archive; (5) on primary bump the old file moves to `<spec-dir>/archive/` with `Status: superseded`. Classification: universal, applicable conditionally per the scope condition.

Cascade requirement: This anchor (verbatim or by `§11.4.73` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a release blocker when a project has a main spec and lets it drift. **Canonical authority:** constitution submodule `Constitution.md` §11.4.73 for the full mandate.

§11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don't-Reimplement (cascaded from constitution submodule §11.4.74)

> Verbatim user mandate (2026-05-20): "*We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules furter! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!*"

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — `vasic-digital` and `HelixDevelopment` on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature's tracker entry with a `Catalogue-Check:` field (`reuse <org/repo>@<sha>` / `extend <org/repo>@<sha>` / `no-match <date>`). Classification: universal.

Cascade requirement: This anchor (verbatim or by `§11.4.74` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. **Canonical authority:** constitution submodule `Constitution.md` §11.4.74 for the full mandate.

§11.4.78 — CodeGraph code-intelligence mandate (cascaded from constitution submodule)

Inherited from `constitution/Constitution.md` §11.4.78. Every project worked on by AI coding agents — and every owned submodule when developed standalone — MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph`, npm package `@colbymchenry/codegraph`): a local SQLite semantic code-knowledge-graph exposed to AI agents over the Model Context Protocol (MCP), 100% local with no cloud or external API. Install globally via npm (no `sudo` — the npm prefix MUST be user-writable). Run `codegraph init` + `codegraph index`: `.codegraph/config.json` is tracked; `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `config.json` `exclude` list MUST exclude other-owned submodules and — non-negotiably — every §11.4.10 credential/secret path. Wire the `codegraph serve --mcp` MCP server into every CLI agent the developers use (Claude Code `.mcp.json`, OpenCode `opencode.json`, Qwen Code `.qwen/settings.json`, Crush `.crush.json`, Kimi CLI `~/.kimi/mcp.json`); every config references the bare `codegraph` command on `PATH`. Cover the integration with an anti-bluff verification suite whose per-agent end-to-end layer uses an unforgeable challenge (a fact obtainable only by calling a CodeGraph MCP tool); un-runnable agents are documented SKIP gaps per §11.4.3, never faked PASSes. Document everything in `docs/CODEGRAPH.md`.

Cascade requirement: this anchor (verbatim or by §11.4.78 ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.md`, and `QWEN.md`. See the constitution submodule `Constitution.md` §11.4.78 for the full mandate. Non-compliance is a process violation.